

Authentifizierter WKD-Datendienst (IT-Sicherheit Aufgabe 03)

Dieses Projekt implementiert einen sicheren **Web Key Directory (WKD)** Server, der um eine kryptographische Zugangskontrolle erweitert wurde. Das System ermöglicht Nutzern, sich **ohne Passwörter**, rein durch den Besitz ihres privaten OpenPGP-Schlüssels, zu authentifizieren (Challenge-Response-Verfahren). Nach erfolgreichem Login erhalten sie Zugriff auf geschützte, persönliche Daten, verwaltet durch ein sicheres Session-Management.

Inhaltsverzeichnis

1. [Funktionsweise & Architektur](#)
2. [Voraussetzungen](#)
3. [Installation & Start](#)
4. [Workflow: Schritt-für-Schritt Anleitung](#)
 - [Schritt 1: Nutzer registrieren \(Keys holen\)](#)
 - [Schritt 2: Authentifizierung \(Challenge-Response\)](#)
 - [Schritt 3: Zugriff auf geschützte Daten](#)
5. [Technische Dokumentation der Komponenten](#)
 - [Java-Server Klassen](#)
 - [Hilfs-Skripte](#)
6. [Protokoll-Details](#)

Funktionsweise & Architektur

Das System setzt sich aus vier logischen Modulen zusammen, die in einem Java-Server integriert sind:

1. **Identifikatoren (Schlüssel-Sammler):** Ein Client-Modul, das Public Keys von anderen WKD-Servern abrufen, deren Vertrauenswürdigkeit (Trust) anhand von CA-Signaturen prüft und sie lokal importiert.
2. **Session Management:** Eine Verwaltung für HTTP-Sitzungen. Es werden kryptographisch starke Session-IDs generiert und über `HttpOnly / Secure Cookies (ITS25-SID)` an den Client gebunden.
3. **Authentifizierung (Challenge-Response):** Ein Login-Verfahren ohne Passwörter. Der Server sendet eine zufällige Challenge (Nonce), die der Client mit seinem privaten PGP-Schlüssel signieren muss.
4. **Zugriffskontrolle (ACL):** Eine Autorisierungslogik, die sicherstellt, dass Nutzer nur auf ihre eigenen Dateien zugreifen dürfen (Owner-Prinzip), während Administratoren (`lars.fischer`) globalen Zugriff haben.

Voraussetzungen

- **Betriebssystem:** Linux (z.B. Debian/Ubuntu im Labor).
- **Java:** JDK 11 oder höher.
- **GnuPG:** `gpg` muss installiert und konfiguriert sein.
- **Tools:** `curl` , `base64` .
- **Netzwerk:** Port `8000` muss verfügbar sein.

Installation & Start

1. Kompilieren

Das `build.sh` Skript kompiliert alle Java-Quellen aus dem Ordner `src/` und legt die Bytecode-Dateien (`.class`) im Ordner `bin/` ab.

```
./build.sh
```

2. Server starten

Startet den `WKDServer` . Der Server lauscht auf Port `8000` und gibt Log-Meldungen auf der Konsole aus.

```
./run.sh
```

(Hinweis: Lassen Sie dieses Terminalfenster offen, um die Server-Aktivitäten zu überwachen.)

Workflow: Schritt-für-Schritt Anleitung

Schritt 1: Nutzer registrieren (Keys holen)

Bevor ein Nutzer (z.B. ein Team-Mitglied) sich einloggen kann, muss der Server dessen öffentlichen Schlüssel kennen und ihm vertrauen.

Aktion: Wir nutzen das Skript `get_key.sh` . Es ist intelligent und erkennt, ob nur ein Nutzernamen oder eine ganze E-Mail übergeben wurde.

```
# Beispiel: Registriere den Nutzer 'kevin.nguefackdjoukeng'  
./get_key.sh kevin.nguefackdjoukeng
```

Hintergrundprozess:

1. Der Key wird vom WKD der Domain `smail.hs-bremerhaven.de` geladen.

2. Der Key wird in den lokalen GPG-Keyring des Servers importiert.
3. Es wird geprüft, ob der Key von einer vertrauenswürdigen CA signiert ist.
4. Eine persönliche Datei (die geschützte Ressource) wird für den Nutzer angelegt.

Schritt 2: Authentifizierung (Challenge-Response)

Da `curl` keine native GPG-Unterstützung hat, führen wir den Login in zwei Phasen durch.

Phase A: Challenge anfordern Wir versuchen, auf die geschützte Ressource zuzugreifen. Da wir noch nicht eingeloggt sind (kein Cookie), lehnt der Server ab und sendet eine Challenge.

```
./1_get_challenge.sh
```

- **Ergebnis:** Das Skript gibt eine kryptische Zeichenkette aus (Base64-kodierte Nonce).
- **Aktion:** Kopieren Sie diese Zeichenkette in die Zwischenablage!

Phase B: Challenge beantworten (Login) Wir signieren die Challenge lokal mit unserem privaten PGP-Schlüssel und senden die Signatur an den Server zurück.

```
# Fügen Sie die kopierte Challenge als Argument ein
./2_send_response.sh "HIER_DIE_KOPIERTE_CHALLENGE_EINFÜGEN="
```

- **Ergebnis:** Bei Erfolg meldet der Server `HTTP 200 OK` und setzt einen Session-Cookie, der im file `cookies.txt` gespeichert wird.

Schritt 3: Zugriff auf geschützte Daten

Jetzt besitzen wir einen gültigen Session-Cookie. Wir können nun auf die geschützte Ressource zugreifen, ohne uns erneut authentifizieren zu müssen.

```
./3_check_login.sh
```